Software Quality Assurance (SQA) Plan

By SCEL Group 2

Version <1.1>

Date: 23 September 2026

Signature Page

Prepared by: Gary Quah	**Date**: **23/09/2026**
Reviewed by: Lee Zheng Wee	**Date**: **23/09/2026**
Reviewed by: Rusyaidi	**Date**: **23/09/2026**
Reviewed by: Quah Yew Yi	**Date**: **23/09/2026**
Reviewed by: Matthew Wong	**Date**: **23/09/2026**
Reviewed by: Lau Jing Jie	**Date**: **23/09/2026**
Approved by: Gary Quah	**Date**: **23/09/2026**

Document Change Record

Revision	Description of Change	Approved by	Date
<0.10>	Filling in of template	Gary	10/09/2026
<0.20>	Review of documentation	Gar	16/09/2026
<1.0>	Final modifications	Gary	23/09/2026
<1.1>	Revised architecture scope to Next.js/Node.js with one FastAPI backend. Separated internal-endpoint and live-AI latency targets, defined p95 measurement over at least 30 requests and distinguished the 10-second timeout.	Gary	04/11/2026

Contents

1. Purpose and Scope 5
    1.1. Purpose 5
    1.2. Scope 6
2. Reference Documents 6
3. Management 6
    3.1. Management Organisation 7
        3.1.1. Project Management 7
        3.1.2. Assurance Management 7
    3.2. Tasks 7
        3.2.1. Product Assessments 8
        3.2.2. Process Assessments 9
    3.3. Roles and Responsibilities 9
        3.3.1. QAM 9
        3.3.2. Software Quality Personnel 10
4. Documents 10
    4.1. Purpose 10
    4.2. Minimum Document Requirements 10
5. Standards, Practices, Conventions and Metrics 11
    5.1. Purpose 11

5.2. Software Quality Programme 11
5.2.1. Standard Metrics 11
6. Software Reviews 13
6.1. Purpose 13
6.2. Minimum Software Reviews 13
7. Test 13
8. Problem Reporting and Corrective Action 14
9. Tools, Techniques and Methodologies 14
9.1. Software Quality Tools 15
9.2. Software Quality Techniques 15
9.3. Software Quality Methodologies 15
10. Media Control 15
11. Record Collection, Maintenance, and Retention 16
12. Training 16
13. Risk Management 17
14. SQA Plan Change Procedure and History 17

1. Purpose and Scope

1.1. Purpose

The purpose of this Software Quality Assurance (SQA) Plan is to establish the goals, processes, and responsibilities required to implement effective quality assurance functions for the <Fitness Tracker> project.

The Software Quality Assurance Plan provides the framework necessary to ensure a consistent approach to software quality assurance throughout the project life cycle. It defines the approach that will be used by the QAM and Software Quality (SQ) personnel to monitor and assess software development processes and products to provide objective insight into the maturity and quality of the software. The systematic monitoring of products, processes, and services will be evaluated to ensure they meet requirements and comply with policies, standards, and procedures, as well as applicable Institute of Electrical and Electronic Engineers (IEEE) and ISO standards.

1.2.Scope

The purpose of SQA is to ensure that the software developed does not deviate from the original intended product. SQA is also concerned to identify any errors, omissions, inconsistencies, and alternatives, enhancements or improvements that can be made at any stage of development.

The project is a smart fitness tracker web application designed to deliver personalized workout routines and nutrition tracking using artificial intelligence and data analytics. The software is developed internally by the project development team. The software items covered by this plan encompasses all components of the system, including a front-end interface developed with Next.js and styled with Tailwind CSS, a Python FastAPI backend for application APIs and in-process readiness scoring, a PostgreSQL database hosted on Supabase, and integration with external services including the Google Gemini API and Strava API. The intended use of this application is to assist users of various fitness levels in tracking biometric metrics, analyzing meal nutrition via image uploads, following adaptive training plans, consulting an AI fitness assistant, and engaging with social gamification features. Node.js is the runtime for the Next.js application.

2. Reference Documents

- IEEE STD 730-2002, IEEE Standard for Software Quality Assurance Plans (http://standards.ieee.org/reading/ieee/std_public/description/se/730-2002_desc.html)
- ISO IEC 90003:2004 Software Standard (http://praxiom.com/iso-90003.htm)
- Project Plan
- System Requirement Specifications

3. Management

This section describes the management organizational structure, its roles and responsibilities, and the software quality tasks to be performed.

3.1. Management Organisation

The implementation of quality assurance system is the responsibility of the Quality Assurance Manager (QAM).

3.1.1. Project Management

The Project Manager will be responsible for approving::

- The system requirement specification document
- The overall time scale for the project
- The choice of system development life cycle
- The choice of software development tools and techniques utilised
- The selection of project teams
- The training of project teams

3.1.2. Assurance Management

The QAM provides Project Management with visibility into the processes being used by the software development teams and the quality of the products being built. The QAM maintains a level of independence from the project and the software developers.

In support of software quality assurance activities, the QAM has assigned and secured Software Quality personnel from the pool of available SQ trainees to coordinate and conduct the SQ activities for the project and report back results and issues.

3.2. Tasks

This section summarizes the tasks (product and process assessments) to be performed during the development of software. These tasks are selected based on the developer's Project Plan and planned deliverables, and identified reviews.

3.2.1. Product Assessments

The following product assessments will be conducted by SQ personnel:

1. User Interface (UI)
    a. UI should be intuitive and user-friendly. Users should be able to navigate the application with ease without much guidance.

2. Settings & Profile Engine (TDEE & BMR Calculation)
    a. Validate the mathematical accuracy of the Mifflin-St Jeor equation to ensure the Basal Metabolic Rate (BMR) and total daily energy expenditure (TDEE) are computed correctly based on the selected activity multiplier by the user.
    b. Ensure that the frontend and backend systems validate users' input and handle invalid biological inputs such as negative weights, out-of-bounds values or missing fields.
    c. Verify that changes to a user's biological data or activity levels are instantly corrected and recalculated to match the changes.

3. AI meal analysis component (Vision confidence and macro aggregation)
    a. Assess system behaviour when LLM vision model returns a low confidence score or fails to identify the image; users should be prompted to manually review and input their meal data
    b. Test the ingestion pipeline using various image formats to ensure that client-side formatting and backend processing handle the payloads without data loss or timeouts

4. Training Routine Generation
    a. Validate training routine strictly adheres to the configured safe threshold based on the user's training history and biological data, preventing the system from recommending dangerous or impossible training load

5. FastAPI Backend endpoints (Strava OAuth 2.0)
    a. Validate that OAuth securely exchanges temporary keys and tokens.
    b. Ensure the backend automatically refreshes keys using refresh_token when user's short lived access token expires

3.2.2. Process Assessments

The following process assessments will be conducted by SQ personnel:

1. Peer Review Process
    a. GitHub repository should enforce a strict requirement of at least one independent approval on every pull request (PR) prior to merging
    b. Ensure that PR descriptions are clearly articulated, reference the exact backlog task and outline the steps performed

2. Maintainability management process
    a. Evaluate the team's adherence to structural separation of concerns; ensure that frontend and backend systems are decoupled
    b. Ensure that the overall codebase adheres to standardised linting rules and contains sufficient documentation to support future scaling and handovers

3.3. Roles and Responsibilities

This section describes the roles and responsibilities for each assurance person assigned to the Project.

3.3.1. QAM

Responsibilities include, but are not limited to:

- Secure and manage SQ personnel resource levels

- Ensure that SQ personnel have office space and the appropriate tools to conduct SQ activities
- Provide general guidance and direction to the SQ personnel responsible for conducting software quality activities and assessments
- Assist SQ personnel in the resolution of any issues/concerns and/or risks identified as a result of software quality activities
- Escalate any issues/concerns/risks to project management

3.3.2. Software Quality Personnel

Responsibilities include, but are not limited to:

- Develop and maintain the project software quality assurance plan
- Generate and maintain a schedule of software quality assurance activities
- Conduct process and product assessments, as described within this plan
- Identify/report findings, observations, and risks from all software assurance related activities to the QAM

4. Documents

4.1. Purpose

This section identifies the minimum documentation governing the requirements, development, verification, validation, and maintenance of software that falls within the scope of this software quality plan. Each document below shall be assessed (reviewed) by SQ personnel.

4.2. Minimum Document Requirements

- System Requirement Specifications
- Quality plan
- Use case diagrams

5. Standards, Practices, Conventions and Metrics

5.1. Purpose

This section highlights the standards, practices, quality requirements, and metrics to be applied to ensure a successful software quality program.

5.2. Software Quality Programme

These practices and conventions are tools used to ensure a consistent approach to software quality for all programs/projects.

1. Scalability: The code should be designed to be scalable to handle an increasing amount of features, users, requests and data without performance degradation.
    a. Do Not Repeat Yourself (DRY)
2. Reusability: Common components that are used throughout the project should be created as reusable features that can be modified in a single location.
3. Maintainability: The code should be designed in a manner where it's easy to maintain by following architecture patterns.
    a. Frontend shall follow the Model View, View Model architecture
    b. Backend shall follow a layered architecture where code is split into categories of: Controllers, services, components, models.
4. Ease of understanding
    a. Code should not be overcomplicated and should be simple, allowing any developer to use or modify code where needed.
    b. Code should include documentation such that any developer can understand the functionality of the code.

5.2.1. Standard Metrics

The following standard metrics are the minimum planned metrics that will be collected, reported, and maintained in the area of software quality assurance:

- API Response Time/ Latency (Performance)

Measure the time from when the backend receives a request to when it returns the complete response. Times are recorded on the server and reported as the 95th percentile (p95) over at least 30 requests per endpoint type, with AI requests using the live Gemini API.

  - Target: Response time (p95) to be under 300ms for internal endpoints (e.g fetching profile, daily logs)
  - Target: Response time (p95) to be under 3s for multimodal AI analysis and LLM chat responses.
  - Timeout: AI requests are cancelled after 10s. This is a timeout limit, not a performance target.

- Code Coverage (Maintainability/ Reliability)

  The percentage of source code branches and lines executed by automated unit and integration test suites across both the frontend and backend

  - Target: Minimum 80% line coverage and 75% branch coverage enforced prior to merge

- Defect Density (Maintainability / Reliability)

  Measure the number of confirmed software defects relative to the size of the entire codebase, quantified as defects per thousand lines of code (KLOC).

  - Target: Threshold of < 2.0 defect/KLOC

- Initial Load Time (User Experience / Performance)

  Measure the time taken from when a user navigates to the application URL until the first piece of meaningful DOM content appears on screen

  - Target: Initial load time < 1.8s

- Test Pass Rate (Maintainability / Reliability)

  Measures the percentage of planned test cases (unit, integration, and manual acceptance tests) that execute successfully without failure prior to submission. This quantifies whether all functional requirements and edge cases are adequately verified

  - Target: 100% pass rate on production releases

- Cyclomatic Complexity (Maintainability/ Scalability)

  Measures the structural complexity of the codebase by counting the number of linearly independent paths through each function or method. SQA tracks this using automated static analysis tools to ensure code remains modular and understandable, enforcing an upper threshold

  - Target: maximum cyclomatic complexity <= 10 per function

6. Software Reviews

6.1. Purpose

This section identifies the number and type of system/subsystem reviews and engineering peer reviews that will be supported by the SQ Personnel. The project milestone chart, and the SQ Personnel resource levels determine the reviews that are supported.

6.2. Minimum Software Reviews

For each review, SQ will assess the review products to assure that review packages are being developed according to the specified criteria, the review content is complete, accurate, and of sufficient detail, and Requests for Action are captured, reviewed, and tracked to closure. In addition, SQ will assess the processes used to conduct the reviews to determine if appropriate personnel are in attendance, correct information is presented, entry and exit criteria are met, and appropriate documents are identified for update.

The following software reviews will be assessed by SQ:

- Project Plan Review
- Requirements Analysis Review
- Software Design Review

- Test Plan Review
- Acceptance Review

7. Test

SQ personnel will assure that the test management processes and products are being implemented per Test Plan. This includes all types of testing of software system components as described in the test plan, specifically during integration testing (verification) and acceptance testing (validation). SQ personnel will monitor testing efforts to assure that test schedules are adhered to and maintained to reflect an accurate progression of the testing activities. SQ will assure that tests are conducted using approved test procedures and appropriate test tools, and that test anomalies are identified, documented, addressed, and tracked to closure. In addition, SQ will assure that assumptions, constraints, and test results are accurately recorded to substantiate the requirements verification/validation status. SQ personnel will review post-test execution related artifacts including test reports, test results, problem reports, updated requirements verification matrices, etc.

In addition, SQ personnel will also ensure that automated unit tests are executed on every pull request before a merge. Test cases are written for, firstly, the readiness score, and secondly, the macronutrient calculation logic before code is being merged.

8. Problem Reporting and Corrective Action

SQ personnel generate, track, and trend assessment findings and observations in a JIRA Reporting and Corrective Action System. Findings and observations are logged as issues in the team's JIRA project where it is labelled and categorised by severity, linking to the affected requirement or module.

Weekly Sprint meetings are set up as well, where team members will take turns sharing their progress as well as addressing problems/challenges faced during each phase/sprint of the project.

Assessment data and corrective action status are reported by the QA manager to the project manager during the sprint review, which will be summarised in the meeting minutes. Any defect or issue classified with high severity will be escalated to the project manager directly instead of waiting for a sprint review.

9. Tools, Techniques and Methodologies

SQ personnel will require access to the following:

9.1. Software Quality Tools

- Microsoft Office tools (i.e., Word, Excel, and PowerPoint)
- Version Control System (eg, Git)
- SonarQube for IDE
- Jira for defect tracking

9.2. Software Quality Techniques

- Requirements Review
- Peer and code reviews
- Functional and regression testing
- Risk-based testing
- Defect tracking and root-cause analysis

9.3. Software Quality Methodologies

- Gitlabs
- GitHub
- Google Docs
- Google Sheets
- Jira

10. Media Control

SQ deliverables will be documented in one of the following Microsoft software applications: Word, Excel, or PowerPoint. Deliverables will be in soft copy, with the exception of completed checklists from process and product assessments. See Section 12 for additional details on the collection and retention of key records. Software Quality personnel will request space on the project's secured server for SQ records. This server is password-protected and backed up nightly.

SQ Records fall under two categories: code and documentation.

a. SQ Records for code are maintained in the team's GitHub repository, serving as the configuration management system of the project. All SQ deliverables including assessment reports and completed checklists will be subject to version control and access is restricted to only team members and lab tutor.
b. SQ Records for documentation are maintained in Google Drive, where documents have version history, ensuring all edits are recorded and reversible.

11. Record Collection, Maintenance, and Retention

SQ personnel will maintain records that document assessments performed on the project. Maintaining these records will provide objective evidence and traceability of assessments performed throughout the project's life cycle. There are two types of records that will be maintained: Hardcopy and Electronic. SQ personnel will maintain electronic or hard copies of all assessment reports and findings. SQ Project folders will contain hard copies of the assessment work products such as completed checklists, supporting objective evidence, and notes.

The table below identifies the record types that will be collected, as well as the Record Custodian and Retention period

Record Title	Record Custodian	Record Retention

SQA Assessments	SQ Personnel	One Year
SQA Checklists	SQ Personnel	One Year
Deliverable Defects	SQ Personnel	One Year

12. Training

SQ personnel have fundamental knowledge in the following areas through prior experience, training, or certification in methodologies, processes, and standards:

· Audits and Reviews (Assessments)

· Risk Management

· Software Assurance

· Configuration Management

· Software Engineering

· ISO 9001, ISO 9000-3

· CMMI

· Verification and Validation

13. Risk Management

SQ personnel will assess the project's risk management process and participate in bi-weekly risk management meetings and report any software risks to the QAM and the project manager.

1. Milestone-Driven Risk Reviews
    a. The QA team will enforce mandatory risk assessment at critical junctions during the development phase. A dedicated review must occur immediately before every milestone. One milestone example would be to validate Next.js frontend before integrating with the FastAPI backend ensuring that all policies are fully aligned

b. The team must evaluate currently unresolved technical debt. If the foundational feature remains unstable or insecure, the project manager must halt the progression to the next SDLC phase until it has been fully resolved.

14. SQA Plan Change Procedure and History

SQ personnel are responsible for the maintenance of this plan. It is expected that this plan will be updated throughout the life cycle to reflect any changes in support levels and SQ activities. Proposed changes shall be submitted to the Quality Assurance Manager (QAM), along with supportive material justifying the proposed change.